Bug Slayers Bot

A step-by-step team guide. A bug ticket goes in; the pull request that most likely caused it, its author, and the code before and after the change come out.

Team Guide · written for new joiners | Python 3.9+ · standard library only | No AI model in the path | 42 tests passing | Every claim checkable with git

Table of contents

1. What this is, in one page

Bug Slayers Bot reads a bug ticket in Jira and posts a comment naming the pull request that most likely caused it, who wrote it, and what the code looked like before and after that change.

It does this by reading git history. Nothing else. There is no machine learning model in the path, no training data, and no external service. Given the same repository and the same ticket, it produces the same answer every time, and every claim it makes can be checked with a `git` command you can run yourself.

Three sentences to remember:

1. A bug report is text. Source code is text. The bot finds which file the text

is talking about.

1. Git knows every change ever made to that file, who made it, and when.
2. The change that best explains the report is the suspect. The bot ranks the

candidates and shows its reasoning.

It is written in Python using only the standard library. There is nothing to `pip install`.

---

2. The problem we are solving

A bug arrives. Before anybody can fix it, somebody has to answer three questions, and answering them is slow and repetitive work:

Which file is this about? The reporter describes a symptom on screen. The engineer has to translate that into a location in a codebase of thousands of files.

What changed there recently? The engineer opens git history for that file and reads through commits looking for something that matches the description.

Who should look at it? Whoever wrote that change, or the team that owns that area.

None of this is difficult. All of it is mechanical, and it happens on every single bug. That is exactly the kind of work a tool should do.

> The bot does not fix bugs and does not decide anything. It does the looking-up so that a human starts from a named pull request instead of from a blank page.

---

3. What the bot posts on a ticket

This is a real comment, posted automatically on a real ticket. Read it before reading how it works, because everything after this is in service of producing this.

```
🤖 Bug Slayers Bot — automated triage, not a human judgement

Suspect change

Most likely: PR #2 — Carol Nair, 2026-08-11
  "chore: shorten the playback session banner wording"

Why:
  - modifies the starting-point file
  - changed code mentions 'hello', 'welcome'
  - landed the same day the bug was reported
  - most recent substantive change to this code

Function changed: announceSessionStart()

The code now:
    public static func announceSessionStart() {
        print("Hello")
    }

The same function before PR #2:
    public static func announceSessionStart() {
        print("Welcome")
    }

Starting point: TASK/TASK/Modules/Player/PlaybackGreeting.swift
Why this file: mentions PlaybackGreeting.

Other candidates:
  - PR #1 — Alice Kumar, 2026-08-11
  - PR #5 — Carol Nair, 2026-06-24

Confidence: High (0.81)
```

Notice what it does and does not say. It names a suspect and shows the evidence. It shows the previous version of the function, taken from git history, so the reader can see what the behaviour used to be. It does not claim to know the fix, and it lists the runners-up so a wrong first guess is not a dead end.

---

4. Words you need first

If you are new, these ten terms are all you need to follow the rest.

TERM	PLAIN MEANING
Commit	One saved change to the code, with an author, a date and a message
Pull request (PR)	A bundle of commits reviewed before joining the main code
Merge commit	The commit git creates when a PR is merged in
Squash merge	A PR merged as one single commit, with the PR number in its message
Revision or ref	A pointer to a version of the code, such as `origin/main`
Diff	The lines added and removed by a commit
Blame	Asking git who last changed each line of a file
JQL	Jira Query Language, the search syntax for finding Jira issues
Idempotent	Safe to run twice; the second run changes nothing
Runner	The temporary machine GitHub gives you to execute a workflow

Two more that are specific to this project:

Seed file, or starting-point file. The one file the bot believes the ticket is about. Everything downstream depends on getting this right.

Localizer. The part that picks the seed file from the ticket's words. It lives in `localize.py`.

---

5. How it works, stage by stage

Seven stages. Each is one Python module, each is independently testable, and each is boring on purpose.

Stage 1 — Get the ticket

`jira_agent.py` asks Jira for issues in scope using JQL, then reads each one's summary and description. Both are used. The title alone is often too short to identify anything.

Stage 2 — Get the code

`repo.py` makes the repository self-provisioning. If it has not been cloned before it is cloned into `~/.cache/bugtrail/repos`; if it has, it is refreshed.

This is why the bot works from anywhere and does not depend on your laptop. It analyses `origin/main` from the remote, not whatever happens to be checked out on somebody's machine.

Stage 3 — Find the file

`localize.py` turns ticket text into a ranked list of candidate files, using three kinds of evidence.

Code-shaped identifiers. Words like `PlaybackGreeting` or `announceSessionStart` that look like symbols. These are the strongest signal.

Distinctive prose words searched inside file contents. If a reporter quotes text they saw on screen, that string is usually sitting in a source file.

Path words. Ordinary words from the report matched against file and directory names, weighted so that a rare word counts for more than a common one.

Rare words score higher than common ones, test files are penalised because fixes usually land in the source file, and the bot's own source is excluded so it cannot blame itself.

Stage 4 — Read the history

`archaeology.py` walks the commits that touched the seed file, follows the file through renames, and maps each commit to a pull request number. It handles both merge styles: `Merge pull request #123` and the squash form `(#123)`.

It also extracts the specific function a commit changed, and can pull that whole function out of any revision. That is what makes the before-and-after possible.

Stage 5 — Score the candidates

`ranking.py` gives every candidate commit a score, explained in section 7, and attaches a plain-English reason to each point it awards.

Stage 6 — Decide whether to speak

If no file was found, or the best score is below the confidence threshold, the bot says so plainly rather than guessing. This is covered in section 17.

Stage 7 — Write the comment

`jira_bot.py` renders the comment in Jira's markup, and the agent posts it. If a comment from the bot already exists, it is updated instead of duplicated.

---

6. How Jira and GitHub fit together

This section answers the question people ask first, and the answer surprises most of them.

Jira does not know the repository exists

There is no integration between the two. Jira has no setting pointing at our code, no plugin installed, and no idea that GitHub is involved at all. It is entirely passive: it stores tickets and answers questions when asked.

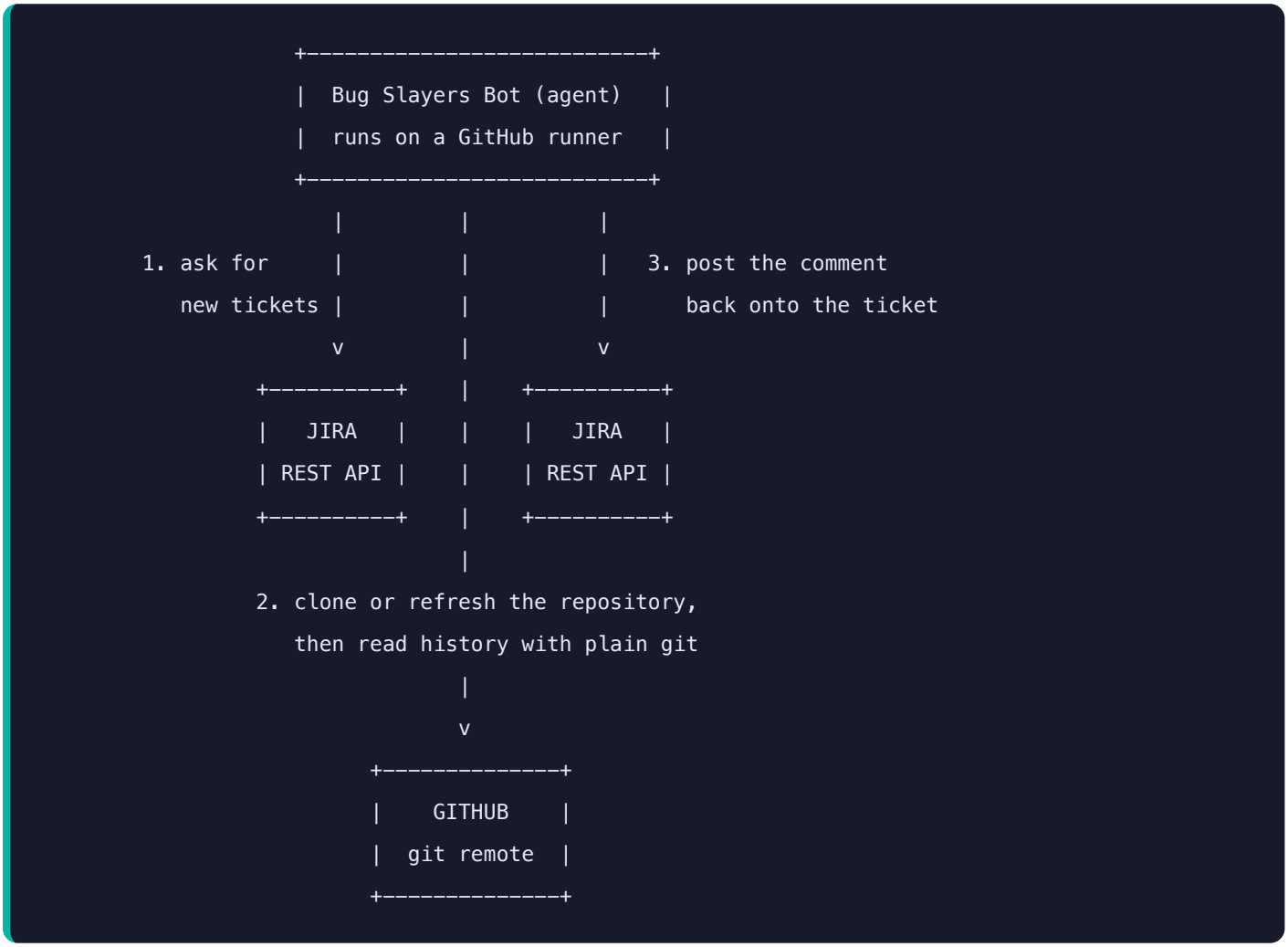
The link between a Jira project and a git repository exists in exactly one place, our own configuration:

```json
{
  "repo": "Monish-WBD/marrow-bugtrail-fixture"
}
```

Change that line and the same tickets are analysed against a different codebase. Nothing in Jira changes, because nothing in Jira ever knew.

Who calls whom

Everything is initiated by the agent. Neither Jira nor GitHub ever calls us.



Step by step, for one ticket:

1. The agent asks Jira for issues matching a JQL query, using the Jira REST API.
2. For each issue it reads the summary and description.
3. It clones or refreshes the repository from GitHub into `~/.cache/bugtrail`.
4. All analysis is local `git` commands against that clone. No API calls.
5. It posts a comment back to the ticket through the Jira REST API.

GitHub plays two separate roles that are easy to confuse. It hosts the code that git clones in step 3, and it separately provides the machine the agent runs on. Those are unrelated. The agent would work identically on a laptop, analysing the same GitHub-hosted repository.

Which credential is used where

CREDENTIAL	USED FOR	WHERE IT LIVES
`JIRA_API_TOKEN`	Reading tickets, posting comments	Repository secret, or your local env file
Git access	Cloning the repository	Public repo needs none; a private one needs a deploy key
`CHAIN_TOKEN`	Letting a watch cycle start its successor	Repository secret

The Jira token is the identity the comment is posted as. That is why the comment opens with a coloured banner naming the bot: Jira attributes the comment to whichever account authenticated, so without the banner it reads as though a colleague wrote it by hand.

Three ways a run begins

Scheduled sweep. GitHub's cron starts the workflow every five minutes. It answers anything unanswered, and restarts the watcher if none is alive.

Resident watcher. A long-running job polling every fifteen seconds, each cycle dispatching its successor before it ends.

Push from Jira. Jira Automation can fire a `repository_dispatch` webhook at GitHub the moment a ticket is created, which is the fastest path of all and the only genuinely event-driven one. We do not use it, because creating an automation rule needs project-admin rights on a shared project that we do not have. The workflow already accepts the `jira-issue` event, so enabling it later is a Jira-side change with no code change here.

> The important consequence of polling rather than being pushed to: the bot can only ever be as current as its last sweep. That is why the interval is fifteen seconds and the safety sweep is five minutes.

---

7. How the ranking actually scores a commit

Four weighted signals, defined in `tools/bugtrail/config.json`:

```json
{
  "weights": {
    "recency": 0.35,
    "keyword": 0.35,
    "seedFile": 0.20,
    "substantive": 0.10
  },
  "minConfidence": 0.35,
  "halfLifeDays": 30,
  "maxSuspects": 3
}
```

Recency, 0.35. A change made just before the report is more suspicious than one from a year ago. Age decays on a 30-day half-life rather than a cliff edge.

Keyword overlap, 0.35. Words shared between the ticket and the changed code or commit message.

Seed file, 0.20. A commit that touched the starting-point file itself scores higher than one that touched a sibling in the same module.

Substantive change, 0.10. Real logic changes rank above formatting, comment edits and whitespace.

On top of the weights sit several corrections, each added because of a real wrong answer during development:

CORRECTION	WHY IT EXISTS
Latest-change bonus	A small later fix was losing to the big original feature commit
Unrelated-module penalty	A recent but irrelevant commit was outranking the real culprit
Cosmetic penalty	Reformatting is rarely the cause of a bug
Revert and bot filtering	Automated commits are noise in attribution

The final number is clamped to 1.0 and reported as a confidence. Below `minConfidence`, which defaults to 0.35, the bot declines to name a suspect.

---

8. Why there is no AI model inside

This is the question you will be asked most often, so here is the reasoning.

Attribution is a factual claim. "PR #2 by Carol Nair changed this line" is either true or false, and git already knows the answer with certainty. A language model asked the same question produces something that reads

like an answer and is sometimes wrong in ways that look identical to being right. For a claim that puts a colleague's name next to a bug, that trade is a bad one.

Everything the bot says can be verified:

```bash
# Every claim in the comment reduces to commands you can run yourself.
git log --follow -- path/to/File.swift
git show <sha> -- path/to/File.swift
git show <sha>^:path/to/File.swift
```

The consequences are practical. It runs in about two seconds, costs nothing per ticket, needs no API key, sends no source code to any third party, and produces identical output for identical input, which is what makes it testable at all.

> This is also the strongest possible position on model-agnostic design. There is no model to be agnostic about, and no prompt to break when a vendor changes a default.

---

9. Set up your machine

You need Python 3.9 or newer and git. That is the whole list.

```bash
python3 --version     # 3.9 or newer
git --version
```

Get the code and confirm the test suite passes before changing anything:

```bash
git clone https://github.com/Monish-WBD/marrow-bugtrail-fixture.git
cd marrow-bugtrail-fixture
python3 -m unittest discover -s tools/bugtrail/tests
```

You should see 42 tests pass in well under a second. If they pass, your environment is correct and every failure from here is about your input rather than your setup.

---

10. Run your first analysis, with no credentials

Start here. This step touches no network service and needs no Jira access, so nothing you do can affect a real ticket.

A bug is just a small JSON file. Create one:

```bash
cat > /tmp/mybug.json <<'EOF'
{
  "key": "DEMO-1",
  "summary": "PlaybackGreeting prints Hello instead of Welcome",
  "description": "On start the greeting should read Welcome, as in the previous build. It now prints Hello."
}
EOF
```

Analyse it:

```bash
python3 tools/bugtrail/cli.py --bug-json /tmp/mybug.json --report
```

You should see something close to this:

```
SEED FILE (derived from the report text)
  TASK/TASK/Modules/Player/PlaybackGreeting.swift
  CODEOWNERS: @marrow-ios-player

LIKELY RELATED CHANGES              confidence: High (0.83)

1.  PR #2   "chore: shorten the playback session banner wording"
    author    Carol Nair
    why
              - modifies the starting-point file
              - changed code mentions 'hello', 'welcome'
              - landed the same day the bug was reported
```

What just happened, in order: the localizer read your text and picked a seed file; the repository was cloned or refreshed; git history for that file was walked; each commit was mapped to a PR and scored; the top suspects were printed with their reasons.

Now try weakening the wording. Replace the summary with "the app shows the wrong message when I start watching" and run it again. The confidence drops, and the starting-point file may change entirely, because the identifier the search depended on is gone. That single experiment teaches more about the tool than reading the source does, and it is the same effect described in section 17.

---

11. Read the output

Four things in the report matter, in this order.

Starting point. If this file is wrong, everything below it is wrong. Check this first when an answer looks strange.

The reasons. Never accept the suspect without reading why it was chosen. The reasons are the audit trail, and they are written to be understood by someone who was not involved.

Confidence. Roughly: above 0.7 is worth acting on, 0.35 to 0.7 is worth reading, below 0.35 is not reported at all.

Other candidates. The runners-up. When the first suspect is wrong, the answer is usually second or third rather than absent.

Useful flags while you are learning:

```bash
python3 tools/bugtrail/cli.py --bug-json /tmp/mybug.json --json
python3 tools/bugtrail/cli.py --bug-json /tmp/mybug.json --report --no-module-expansion
python3 tools/bugtrail/cli.py --bug-json /tmp/mybug.json --report --history-limit 200
```

`--json` gives machine-readable output for scripting. `--no-module-expansion` restricts the search to the seed file alone, which is the fastest way to tell whether a wrong answer came from the localizer or from the ranker.

---

12. Connect it to Jira

Jira Cloud does not accept a password over its REST API. You need an API token, which is free and takes a minute to create.

Go to `id.atlassian.com/manage-profile/security/api-tokens`, create a token, and copy it immediately, because it is shown only once.

Store the three values outside the repository so they can never be committed:

```bash
mkdir -p ~/.config/bugtrail
cat > ~/.config/bugtrail/env <<'EOF'
export JIRA_BASE_URL="https://your-site.atlassian.net"
export JIRA_EMAIL="you@company.com"
export JIRA_API_TOKEN="paste-your-token-here"
EOF
chmod 600 ~/.config/bugtrail/env
```

Load them into your shell before running the agent:

```bash
set -a && . ~/.config/bugtrail/env && set +a
```

> ⚠️ Never paste a token into a chat window, a ticket, a screenshot or a commit. If one is ever exposed, revoke it at the same page you created it.

---

13. Shadow mode, then posting for real

The agent does not post unless you ask it to. Run it without `--post` first and read what it would have said:

```bash
python3 tools/bugtrail/jira_agent.py \
    --parent PLAY-126471 \
    --issue-types "Sub-task" \
    --since-days 7 \
    --once
```

When the output looks right, add `--post`:

```bash
python3 tools/bugtrail/jira_agent.py \
    --parent PLAY-126471 \
    --issue-types "Sub-task" \
    --since-days 7 \
    --once --post
```

Scope is deliberately narrow, and there are four ways to set it:

FLAG	MEANING
`--parent`	Only children of one issue. Safest, used for the demo
`--label`	Any issue carrying a label. Opt-in by the reporter
`--project`	A whole project key. Wide; use with care
`--jql`	Extra JQL, combined with the above

`--issue-types` defaults to `Bug`, because attribution is meaningless for a Story or a Task. Posting twice is not a risk: the bot recognises its own previous comment by its signature and updates rather than duplicating.

---

14. Watch mode

Watch mode polls on an interval and comments as tickets appear. This is what you want for a live demonstration, because everything happens in front of you:

```bash
python3 tools/bugtrail/jira_agent.py \
    --parent PLAY-126471 \
    --issue-types "Sub-task" \
    --watch --interval 15 --post
```

Typical time from a reporter pressing Create to the comment appearing is fifteen to thirty seconds, of which nearly all is waiting for the next poll. The analysis itself takes about two seconds.

`--interval` implies `--watch`. That was a real bug once: asking for an interval without the flag produced one healthy-looking sweep and then a silent exit, which is the worst failure available, because a watcher that has stopped looks exactly like a watcher with nothing to do.

---

15. Running it in the cloud

Watch mode needs your laptop awake. For a bug filed at 2am from another timezone, the work belongs on a machine that is always on, so the same agent runs as a GitHub Actions workflow in `.github/workflows/bugtrail.yml`.

Add four repository secrets under Settings, then Secrets and variables, then Actions:

SECRET	PURPOSE
`JIRA_BASE_URL`	Your Jira site URL
`JIRA_EMAIL`	The account the comments are posted as
`JIRA_API_TOKEN`	The token from section 12
`CHAIN_TOKEN`	A GitHub token, needed only for the resident watcher

There are three ways it runs, and they cover each other:

Scheduled sweep, every five minutes. The safety net. It also checks whether a watcher is alive and starts one if not, which is what stops a dead watcher staying dead.

Resident watcher. Fifty-five minute cycles, each dispatching its own successor, giving fifteen-second response. A job cannot outlive its runner, so "permanent" here means a chain of cycles.

On request. For a demo or a test session, without leaving anything running:

```bash
gh workflow run BugTrail -f watch_minutes=20
```

`CHAIN_TOKEN` cannot be the built-in `GITHUB_TOKEN`. Events raised with the built-in token deliberately do not start new workflow runs, so the chain needs a token of its own.

---

16. Prove it works

Three levels of evidence, from fastest to most convincing.

Unit tests. Forty-two of them, covering ranking, comment rendering, idempotency markers and function extraction across brace and indentation languages.

```bash
python3 -m unittest discover -s tools/bugtrail/tests
```

A synthetic repository with known culprits. The fixture repo contains deliberately planted bugs where the correct answer is known in advance, so the ranker can be scored rather than eyeballed.

Real history. `mine_szz.py` derives ground truth from real repositories using the SZZ method: find the commit that fixed a bug, walk back to the commit that introduced the fixed lines. `eval_localize.py` then scores the localizer against it.

```bash
python3 tools/bugtrail/eval_localize.py --repo ../some-repo --tickets tickets.json
```

The evaluation reports three accuracies, and the distinction matters. Exact file is the harshest. Same directory means module expansion would reach the fix from where it pointed. Same library means it identified the right component and therefore the right owning team, which is what triage actually needs.

---

17. When it says nothing, and why that is deliberate

The bot fails closed. Rather than inventing a plausible suspect, it posts a note saying it could not attribute the change and why.

This happens in two situations. Either no candidate file was located, meaning nothing in the ticket matched anything in the code, or a file was found but no change to it stood out above the confidence threshold.

Both notes are provisional. A later run that can do better replaces the note automatically, but a real answer is never overwritten by a worse one.

The commonest cause of "no candidate file located" is a ticket written entirely in symptoms, with no term that appears anywhere in the source. Compare these two reports of the same bug:

```
Weak:   The app shows the wrong message when I start watching.

Strong: PlaybackGreeting prints "Hello" on session start.
        Expected "Welcome", as in build 4.2.
```

Both are valid bug reports. Only the second contains a string the bot can find in the repository. Quoting the exact log line, error text, or on-screen wording is the single most useful thing a reporter can do.

18. Troubleshooting

SYMPTOM	CAUSE AND FIX
`no candidate file located`	Ticket has no term appearing in the code. Quote exact text
Comment never appears	Check scope. Wrong parent or issue type filters it out
Comment appears once, never updates	By design. Use `--update` to refresh
Suspect is the original author	The feature commit outranked a later small fix. Check other candidates
`search failed: 410`	Jira removed the old search endpoint. Update to the v3 path
Watcher exits immediately	Check credentials are loaded into the shell
PR numbers missing, raw SHAs shown	The clone is stale or the merge commit is not on the analysed ref
Cloud run queued for a long time	GitHub is not allocating a runner. The five-minute sweep covers it

19. Known limitations

Be able to state these before somebody else does.

It only sees what is in git. A bug caused by configuration, a backend change, a feature flag or a third-party SDK has no commit to find.

It relies on the ticket containing a findable term. Pure prose defeats it, which is why it reports the dead end rather than guessing.

Exact-file accuracy is much lower than module-level accuracy. It is better understood as "which area and which team" than "which line".

The original author of a feature can outrank the author of a later regression, because the first commit usually contains more matching words.

It suggests. It does not decide. Every comment says so, and that wording is not decoration.

20. Where to change what

YOU WANT TO CHANGE	EDIT THIS
How files are found from ticket text	`localize.py`
How git history is read, PR mapping	`archaeology.py`
How suspects are scored	`ranking.py` and `config.json`
What the Jira comment looks like	`jira_bot.py`
Polling, scope, posting behaviour	`jira_agent.py`
Cloning and refreshing repositories	`repo.py`
The full local report	`report.py`
The drafted regression test	`testgen.py`
Owning-team lookup	`codeowners.py`
Cloud schedule and triggers	`.github/workflows/bugtrail.yml`

Tuning values live in `config.json`, so changing behaviour usually does not mean changing code:

JSON

```
{
  "historyLimit": 60,
  "expandToModule": true,
  "halfLifeDays": 30,
  "moduleWeight": 0.5,
  "cosmeticPenalty": 0.75,
  "minConfidence": 0.35,
  "maxSuspects": 3,
  "excludeReverts": true
}
```

21. Frequently asked questions

Does it need my laptop running? No. The cloud workflow does the same work on GitHub's machines. Watch mode from a terminal is for demos and development.

Will it comment twice if it runs twice? No. It recognises its own signature on a ticket and updates instead.

Can it comment on the wrong project? Only if you scope it that way. Scope is applied to every run regardless of what triggered it, so a dispatch naming an unrelated ticket is filtered out by the same rule a sweep uses.

Does it send our code anywhere? No. It runs git commands locally and calls only the Jira API, to read tickets and post comments.

Does it work for Android as well as iOS? Yes. Swift, Kotlin, Java, Objective-C, Python, TypeScript, JavaScript, Go, Ruby and C# are all indexed.

What if it accuses the wrong person? It is a suggestion with visible reasoning and named runners-up, and the comment says so. A wrong first guess costs the reader a few seconds, not a wrong fix.

Can we point it at a different repository? Yes. Set `repo` in `config.json` or pass `--repo`. It accepts a local path, a clone URL, or `owner/name`.

How do I stop it? Disable the workflow in the GitHub Actions tab. For a local watcher, press Ctrl-C.